

POSTGRADUATE ENTRANCE EXAM • 408

计算机考研 408 数据结构专题

查找与排序 力扣题单精讲

SEARCH & SORTING • LEETCODE PROBLEM LIST

本册以考研 408 数据结构大纲为骨架，聚焦「查找」与「排序」两大高频考点，配套精选力扣题单、图解原理、复杂度分析与解题提示，帮助考生在算法实现与真题应试之间建立完整闭环。

8

核心算法图解

40+

力扣精选题目

2

大章节 • 12 小节

PREPARED FOR
计算机考研 408 备考同学

DATE
2026 年 6 月

目录

第一章 查找 (Search)	4
1.1 顺序查找 (Sequential Search)	4
1.2 折半查找 (Binary Search)	5
1.3 分块查找 (Block Search / 索引顺序查找)	6
1.4 二叉排序树 (BST) 与平衡二叉树 (AVL)	7
1.5 B 树与 B+ 树	8
1.6 散列表 (Hash Table)	10
1.7 查找章节力扣题单汇总	11
第二章 排序 (Sorting)	13
2.1 插入排序	13
2.2 交换排序	14
2.3 选择排序	16
2.4 归并排序 (Merge Sort)	18
2.5 基数排序 (Radix Sort)	20
2.6 外部排序 (External Sort)	20
2.7 排序章节力扣题单汇总	21
第三章 综合练习与真题对照	22
3.1 高频真题题型归纳	22
3.2 算法设计题答题模板	22
3.3 综合练习题 (模拟真题)	22
附录 排序算法复杂度速查表	24

A.2 查找算法 ASL 速查表 24

A.3 关键公式与结论速记 24

前言：408 数据结构考查要点与力扣练习价值

计算机考研 408 统考中，数据结构占据 45 分（共 150 分），是四门科目中分值最高、也最强调“动手实现”的一门。其中**查找与排序**两大板块既是历年真题的高频考点，也是选择题、综合应用题（算法设计题）的常客。从近十年真题分布看，查找与排序合计约占数据结构分值的 35%–45%，几乎每年都会出现至少一道算法设计题或综合性选择题。

力扣（LeetCode）作为全球最大的算法练习平台，其题库覆盖了 408 大纲要求的全部查找与排序算法。然而力扣题库浩如烟海，盲目刷题不仅效率低下，还容易偏离考研方向。本册以 408 考纲为骨架，从力扣题库中精选 40 余道与考纲高度契合的题目，按知识点归类，配套图解原理、复杂度分析、解题提示，帮助考生在“理解原理 → 动手实现 → 真题应试”之间建立完整闭环。

本册使用方法

- **第一遍**：先读知识点讲解与图解，理解算法原理与流程，对照代码模板手写一遍。
- **第二遍**：按题单顺序在力扣上提交，先做“简单”题巩固基础，再做“中等”题强化应用。
- **第三遍**：对“困难”题与综合题，结合 408 真题思路，思考边界条件与复杂度优化。
- **冲刺阶段**：用附录的复杂度速查表快速回顾，确保每种算法的时间/空间复杂度、稳定性、适用场景烂熟于心。

408 应试 vs 力扣刷题 的关键差异

力扣追求“通过所有测试用例”，而 408 算法题要求“写出完整可读的 C 语言伪代码”，评分标准包括：**算法思想（4 分）+ 代码实现（8 分）+ 复杂度分析（1 分）**。因此练习时不仅要 AC，更要训练自己用简洁的 C 风格代码表达算法，并主动写出时间/空间复杂度。

第一章 查找 (Search)

查找是数据结构中最基础也最高频的操作之一。408

考纲要求掌握顺序查找、折半查找、分块查找、二叉排序树与平衡二叉树 (AVL)、B 树与 B+ 树、散列表等查找方法，并能分析各自的平均查找长度 (ASL) 与适用场景。查找算法的核心评价指标是平均查找长度

ASL，它直接决定了算法的效率。在考研真题中，查找常与排序结合考查（如散列表的冲突解决、BST 的构造与平衡），也常以“给定关键字序列，画出查找结构并计算 ASL”的形式出综合题。

1.1 顺序查找 (Sequential Search)

顺序查找是最简单的查找算法：从表的一端开始，逐个比较关键字与给定值，直到找到或扫描完毕。它对表的存储结构无要求（顺序表或链表均可），对关键字是否有序也无要求。虽然效率不高，但它是其他查找算法的基础，也是理解 ASL 概念的最佳切入点。

算法实现 (带哨兵)

经典实现技巧是在数组下标 0 处放置“哨兵”（即待查关键字本身），这样可以从后向前扫描，每次循环只需比较一次（无需判断下标越界），将循环内的比较次数从 2 次降为 1 次，效率提升约一倍。

C · 顺序查找 (哨兵法)

```
typedef struct {
    int *elem; // 数据元素基址, 0 号单元留空作哨兵
    int length;
} SSTable;

int Search_Seq(SSTable ST, int key) {
    ST.elem[0] = key; // 哨兵
    int i = ST.length;
    while (ST.elem[i] != key) // 无需判断 i <= 0
        i--;
    return i; // 返回 0 表示未找到
}
```

性能分析

对于长度为 n 的表，查找成功时：ASL = $(n+1)/2$ （等概率情况下，平均比较次数）。查找失败时：ASL = $n+1$ 。时间复杂度为 $O(n)$ 。当各关键字查找概率不等时，可将高概率关键字放在表尾（从后向前扫描时优先命中），以降低 ASL。

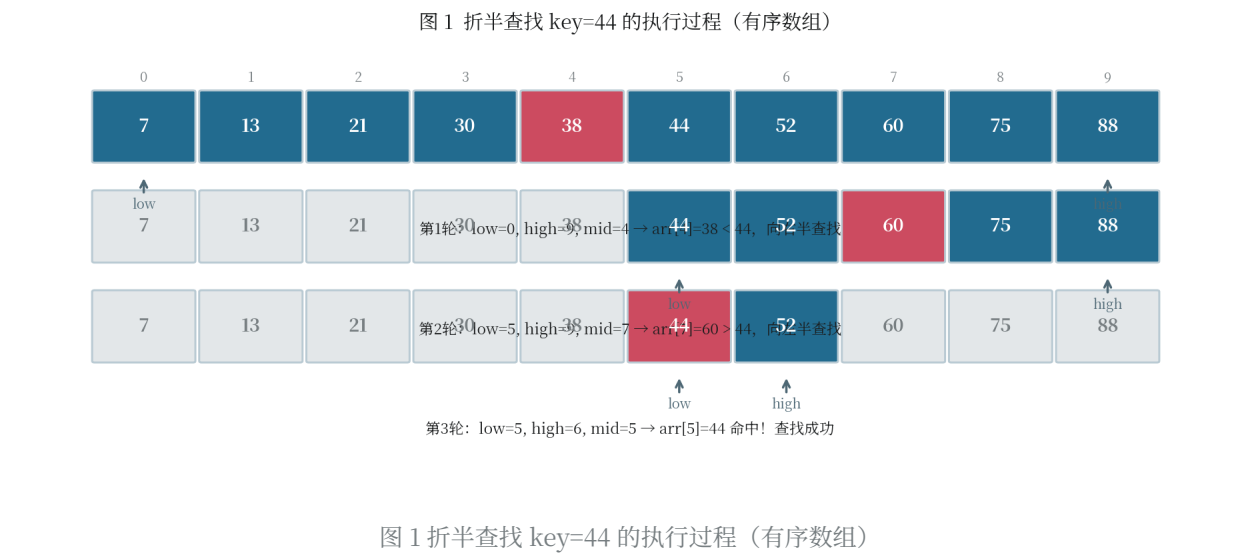
考点提示

顺序查找的“哨兵”思想是 408

选择题的高频考点。务必理解：哨兵的作用是免去循环内的越界判断，从而将每次循环的比较次数从 2 次降为 1 次。注意哨兵法要求 0 号单元空闲，因此数组长度需为 $n+1$ 。

1.2 折半查找（Binary Search）

折半查找（又称二分查找）要求表必须是**顺序存储且关键字有序**。其核心思想是：每次与中间元素比较，若不等则将查找区间缩小一半。折半查找的效率远高于顺序查找（ $O(\log n)$ vs $O(n)$ ），但代价是要求有序且只能顺序存储（链表无法高效随机访问）。



算法实现

C · 折半查找（迭代版）

```
int Binary_Search(int *A, int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2; // 向下取整
        if (A[mid] == key)
            return mid; // 命中
        else if (A[mid] > key)
            high = mid - 1; // 左半区
        else
            low = mid + 1; // 右半区
    }
    return -1; // 未找到
}
```

判定树与 ASL

折半查找的过程可用一棵**判定树（比较树）**描述：根节点是第一次比较的 mid，左右子树对应左右半区。判定树的形态**仅取决于表长 n，与关键字值无关**。对于 n 个元素的有序表，判定树的高度为 $\lceil \log_2 n \rceil + 1$ ，查找成功时的 $ASL \approx \log_2(n+1) - 1$ ，查找失败时的比较次数等于从根到某外部节点的路径长度。

易错点

① **mid 的取整方向**：当 low+high 为奇数时，mid 向下取整（C 语言整数除法自动向下取整）。这会影响判定树的形态，但不影响正确性。② **low 与 high 的更新**：必须是 mid±1，而非 mid。若写成 low=mid 或 high=mid，当区间只剩 2 个元素时会陷入死循环。③ **折半查找不适用于链表**，因为链表无法 O(1) 随机访问中间节点。

力扣题单 · 折半查找

题号	题目	难度	对应考点	解题提示
704	二分查找	简单	折半查找基础模板	标准模板，注意边界与 mid 取整
35	搜索插入位置	简单	折半查找变体	返回 left 即插入位置
34	在排序数组中查找元素第一个和最后一个位置	中等	折半查找 · 左右边界	两次二分：找左边界与右边界
69	x 的平方根	简单	折半查找应用	二分答案，注意溢出用 long
153	寻找旋转排序数组中的最小值	中等	折半查找 · 旋转数组	比较 mid 与 right 决定方向
33	搜索旋转排序数组	中等	折半查找 · 旋转数组	先判断哪半有序再决策
162	寻找峰值	中等	折半查找 · 二分答案	比较 mid 与 mid+1 决定爬坡方向
4	寻找两个正序数组的中位数	困难	折半查找 · 划分	对较短数组二分找划分点

1.3 分块查找（Block Search / 索引顺序查找）

分块查找是顺序查找与折半查找的折中方案。它将表分成若干块，**块间有序、块内无序**：即第 i 块的所有关键字都小于第 i+1 块的最小关键字（或大于最大关键字）。为此需建立一个索引表，索引项包含每块的**最大关键字**和**起始地址**。

查找过程

- **第一步**：在索引表上折半查找（或顺序查找），确定目标关键字所在的块。
- **第二步**：在块内顺序查找目标关键字。

ASL 分析

设表长 n，分为 b 块，每块 s 个元素（n = b×s）。若索引表用顺序查找、块内也用顺序查找，则：ASL = (b+1)/2 + (s+1)/2 = (s² + 2s + n)/(2s)。当 s = √n 时，ASL 取最小值 √n + 1，这是分块查找的最佳块大小。若索引表用折半查找，则 ASL ≈ log₂(b+1) + (s+1)/2。

考点提示

分块查找的"块间有序、块内无序"是核心特征，常考选择题。注意：分块查找的索引表是**有序**的，因此索引表上可用折半查找；但块内是无序的，只能顺序查找。ASL 介于顺序查找和折半查找之间。

1.4 二叉排序树 (BST) 与平衡二叉树 (AVL)

二叉排序树定义与操作

二叉排序树 (BST) 或者是一棵空树，或者是满足以下性质的二叉树：①

若左子树非空，则左子树上所有节点值均小于根节点值；②

若右子树非空，则右子树上所有节点值均大于根节点值；③ 左右子树本身也是二叉排序树。**中序遍历 BST 得到有序序列**，这是 BST 最重要性质。

图 2 二叉排序树 (BST) 的构造与查找路径

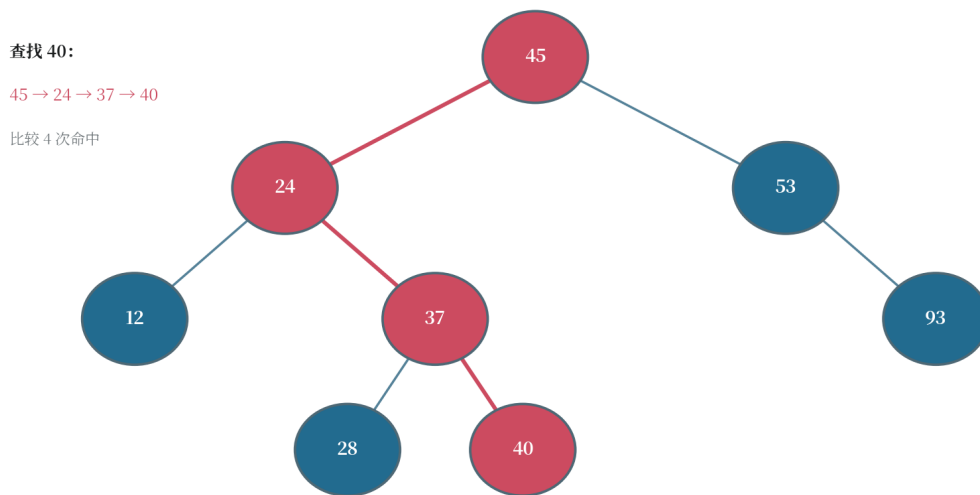


图 2 二叉排序树 (BST) 的构造与查找路径

BST 的查找从根开始，比根小走左子树，比根大走右子树，直到命中或走到空。插入操作类似查找：

找到待插入位置（叶子节点的子位置）后挂接新节点。删除操作分三种情况：①

删除叶子节点直接删；② 删除只有一个子节点的节点，用子节点替代；③ 删除有两个子节点的节点，

用其中序前驱（左子树最大值）或中序后继（右子树最小值）替代后再删除前驱/后继。

平衡二叉树 (AVL)

BST 的形态取决于插入顺序，最坏情况退化为链表（ASL $= O(n)$ ）。平衡二叉树 (AVL) 通过限制任意节点的左右子树高度差不超过 1，保证树高为 $O(\log n)$ ，从而保证查找、插入、删除均为 $O(\log n)$ 。当插入破坏平衡时，需通过**旋转**调整：

失衡类型	触发条件	调整方式	旋转次数
LL（左左）	在左孩子的左子树上插入	右旋（LL 旋转）	1 次
RR（右右）	在右孩子的右子树上插入	左旋（RR 旋转）	1 次
LR（左右）	在左孩子的右子树上插入	先左旋后右旋	2 次
RL（右左）	在右孩子的左子树上插入	先右旋后左旋	2 次

AVL 调整口诀

"左左右旋，右右左旋，左右先左后右，右左先右后左"。判断失衡类型时，从插入点向上找到第一个平衡因子绝对值超过 1 的节点（失衡节点），再看插入点在失衡节点的哪一侧的哪一侧，即可确定类型。

力扣题单 · BST 与 AVL

题号	题目	难度	对应考点	解题提示
700	二叉搜索树中的搜索	简单	BST 查找	迭代或递归，比根小走左
701	二叉搜索树中的插入操作	中等	BST 插入	找到叶子位置挂接
450	删除二叉搜索树中的节点	中等	BST 删除	分三种情况处理
98	验证二叉搜索树	中等	BST 性质	中序遍历判断是否有序
108	将有序数组转换为二叉搜索树	简单	BST 构造 · 平衡	取中点为根递归构造
109	有序链表转换二叉搜索树	中等	BST 构造 · 平衡	快慢指针找链表中点
235	二叉搜索树的最近公共祖先	简单	BST 性质	利用大小关系判断方向
110	平衡二叉树	简单	AVL 平衡判断	递归求高度并判断
1382	将二叉搜索树变平衡	困难	AVL 重平衡	中序遍历 + 108 思路

1.5 B 树与 B+ 树

B 树（B-tree）是一种多路平衡查找树，专为磁盘等外存设备设计。与 BST/AVL 不同，B 树的每个节点可包含多个关键字和多个子指针，从而降低树高，减少磁盘 I/O 次数。408 考纲要求掌握 m 阶 B 树的定义、插入与删除过程，以及 B+ 树与 B 树的区别。

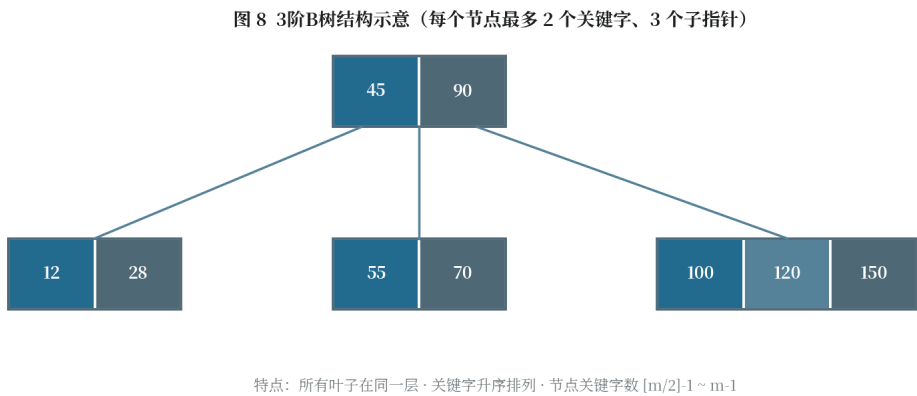


图 8 3 阶 B 树结构示意图（每个节点最多 2 个关键字、3 个子指针）

m 阶 B 树性质

- 每个节点最多有 **m 棵子树**（m 个分支）。
- 根节点至少有 2 棵子树（除非它是叶子）。
- 非根非叶节点至少有 $\lfloor m/2 \rfloor$ 棵子树。
- 节点关键字数范围： $\lfloor m/2 \rfloor - 1 \sim m - 1$ 。
- 所有叶子节点在同一层（绝对平衡），且不带信息（外部节点）。
- 节点内关键字升序排列，且关键字左子树所有值 < 该关键字 < 右子树所有值。

插入与删除

插入：在叶子节点插入。若插入后节点关键字数超过 $m-1$ ，则发生**分裂**：取中间关键字（第 $\lfloor m/2 \rfloor$ 个）提升到父节点，左右两部分成为两个独立节点。分裂可能向上传播。**删除：**若删除后节点关键字数低于 $\lfloor m/2 \rfloor - 1$ ，则需**借**（向兄弟借）或**合并**（与兄弟合并）。若被删关键字在内部节点，用其前驱或后继替代后再删除前驱/后继。

B 树 vs B+ 树

对比维度	B 树	B+ 树
节点存储	非叶节点存关键字 + 数据指针	非叶节点只存关键字（索引）
数据位置	所有节点都可能存数据	数据只在叶子节点
叶子链表	叶子之间无链接	叶子通过链表相连（支持顺序遍历）
查找路径	可能在非叶节点命中即终止	必须查到叶子节点
关键字数	n 个关键字有 n+1 个子树	n 个关键字有 n 个子树
适用场景	文件系统、随机查找	数据库索引、范围查询

高频考点

① **B 树的高度公式**：含 n 个关键字的 m 阶 B 树，高度 h 满足 $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ 。② **B+ 树的叶子链表**是范围查询高效的关键，这是数据库索引选用 B+ 树而非 B 树的核心原因。③ 计算 B 树节点数时，注意根节点至少 2 个关键字、非根至少 $\lceil m/2 \rceil - 1$ 个。

1.6 散列表（Hash Table）

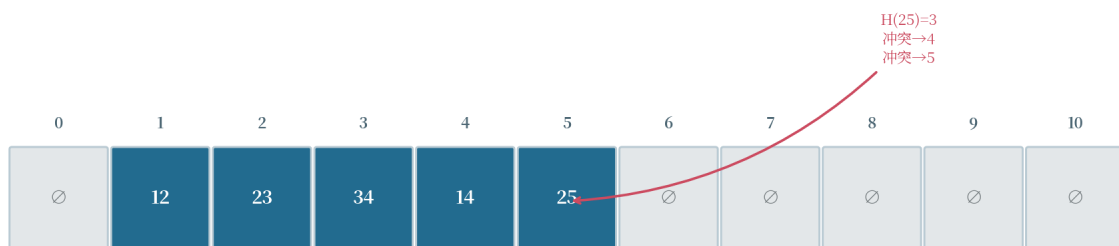
散列表（哈希表）通过**散列函数**将关键字直接映射到存储地址，理想情况下查找时间为 $O(1)$ 。它是 408 的**必考重点**，几乎每年都会出综合题，要求构造散列表、计算 ASL、分析冲突解决策略。散列表的核心矛盾是：**冲突不可避免**（除非关键字集合有限且散列函数完美），因此需要冲突解决方法。

常用散列函数

- **直接定址法**： $H(key) = a \times key + b$ 。无冲突，但要求关键字连续，适用面窄。
- **除留余数法**： $H(key) = key \bmod p$ 。最常用， p 选不大于表长的最大素数。
- **数字分析法**：取关键字中分布均匀的若干位作地址。适合关键字位数多且已知分布。
- **平方取中法**：取关键字平方后的中间几位。关键字位数较少时常用。

冲突解决方法

图 3 散列表冲突解决：线性探测法（Linear Probing）



散列函数 $H(key) = key \bmod 11$ | 插入序列：12, 23, 34, 14, 25

图 3 散列表冲突解决：线性探测法（Linear Probing）

方法	原理	优点	缺点
线性探测法	$H_i = (H(key) + i) \bmod m$	实现简单	易产生"堆积"（聚集）
平方探测法	$H_i = (H(key) \pm i^2) \bmod m$	缓解堆积	可能探测不到所有位置
再散列法	$H_i = RH_i(key)$	不易堆积	计算开销大
链地址法	同义词挂同一链表	无堆积，删除方便	需额外指针空间

ASL 分析（重点）

散列表的 ASL 取决于冲突解决方法与装填因子 $\alpha = \text{表中记录数} / \text{表长}$ 。 α 越大，冲突越频繁，ASL 越大。这是散列表性能的核心指标。对于**线性探测法**，查找成功 $ASL \approx (1 + 1/(1-\alpha))/2$ ，查找失败 $ASL \approx (1 + 1/(1-\alpha)^2)/2$ 。对于**链地址法**，查找成功 $ASL \approx 1 + \alpha/2$ ，查找失败 $ASL \approx \alpha + e^{(-\alpha)}$ 。

真题套路

散列表综合题的标准流程：① 给定散列函数与表长，依次插入关键字，画出散列表；② 计算每个关键字的比较次数；③ 求查找成功 $ASL = \Sigma(\text{比较次数})/\text{关键字数}$ ；④ 求查找失败 $ASL = \Sigma(\text{比较次数})/\text{散列地址数}$ 。注意：查找失败的比较次数是从该散列地址出发到遇到“空”为止的探测次数。

力扣题单 · 散列表

题号	题目	难度	对应考点	解题提示
1	两数之和	简单	哈希表 · 查找	用哈希表存值到下标映射
217	存在重复元素	简单	哈希集合	set 判重
349	两个数组的交集	简单	哈希集合	两 set 求交
350	两个数组的交集 II	简单	哈希表 · 计数	Counter 统计频次
205	同构字符串	简单	哈希表 · 双向映射	两方向各建一张表
146	LRU 缓存	中等	哈希表 + 双向链表	哈希定位 + 链表维护顺序
460	LFU 缓存	困难	哈希表 · 双层结构	频次表 + 节点表
128	最长连续序列	中等	哈希集合 · $O(n)$	只从序列起点开始扩展

1.7 查找章节力扣题单汇总

本节将第一章涉及的力扣题目按知识点重新归类，方便按专题刷题。建议按“折半查找 → BST/AVL → 散列表”的顺序推进，每个专题先做 2-3 道简单题熟悉模板，再做中等题强化变体，最后挑战困难题。

知识点	题号清单	建议优先级
折半查找	704, 35, 34, 69, 153, 33, 162, 4	★★★★★ 必刷
分块查找	(力扣无直接对应题, 理解原理即可)	★★ 理解
BST 基础	700, 701, 450, 98, 235	★★★★★ 必刷
BST 构造	108, 109	★★★★ 推荐
AVL 平衡	110, 1382	★★★★ 推荐
B 树/B+ 树	(力扣无对应题, 掌握手算与原理)	★★ 理解
散列表基础	1, 217, 349, 350, 205	★★★★★ 必刷
散列表进阶	146, 460, 128	★★★★ 推荐

第二章 排序 (Sorting)

排序是数据结构中算法最密集的章节。408 考纲要求掌握插入排序（直接插入、折半插入、希尔）、交换排序（冒泡、快速）、选择排序（简单选择、堆）、归并排序、基数排序七大内部排序算法，以及外部排序的基本思想。每种算法都需掌握其时间复杂度、空间复杂度、稳定性、适用场景，并能手工模拟排序过程。排序是算法设计题的高频考点，尤其是快速排序的划分与堆排序的调整。

2.1 插入排序

直接插入排序

直接插入排序的思想类似整理扑克牌：将待排序序列分为有序区和无序区，每次从无序区取一个元素，在有序区从后向前比较找到合适位置插入。初始时有序区只有第一个元素。对于基本有序的序列，直接插入排序效率很高（接近 $O(n)$ ）。

C · 直接插入排序

```
void InsertSort(int *A, int n) {
    int i, j, temp;
    for (i = 1; i < n; i++) { // A[0..i-1] 已有序
        if (A[i] < A[i-1]) { // 需要插入
            temp = A[i];
            for (j = i - 1; j >= 0 && A[j] > temp; j--)
                A[j+1] = A[j]; // 后移
            A[j+1] = temp; // 插入
        }
    }
}
```

性能：最好 $O(n)$ （已有序），平均/最坏 $O(n^2)$ 。空间 $O(1)$ 。**稳定。**适用于顺序表和链表， n 较小或基本有序时效率高。

折半插入排序

折半插入排序是对直接插入排序的优化：在有序区查找插入位置时用**折半查找**代替顺序查找，将比较次数降为 $O(\log n)$ 。但元素移动次数不变（仍为 $O(n)$ ），因此总时间复杂度仍为 $O(n^2)$ 。折半插入排序**仅适用于顺序表**（链表无法折半）。

希尔排序 (Shell Sort)

希尔排序（缩小增量排序）是直接插入排序的改进版。它先将序列按某个增量 d 分组，对每组分别直接插入排序；然后缩小增量重复，直到 $d=1$ （即对整个序列做一次直接插入排序）。希尔排序利用了“直接插入排序在基本有序时效率高”的特点，通过先做粗粒度排序使序列趋于有序。

C · 希尔排序（增量折半）

```
void ShellSort(int *A, int n) {
    int dk, i, j, temp;
    for (dk = n / 2; dk >= 1; dk = dk / 2) { // 增量序列
        for (i = dk; i < n; i++) {
            if (A[i] < A[i-dk]) {
                temp = A[i];
                for (j = i - dk; j >= 0 && A[j] > temp; j -= dk)
                    A[j+dk] = A[j];
                A[j+dk] = temp;
            }
        }
    }
}
```

希尔排序要点

① 希尔排序**不稳定**（相同关键字可能分到不同组）。② 时间复杂度取决于增量序列，最坏 $O(n^2)$ ，平均约 $O(n^{1.3})$ 。③ 增量序列最后一个值必须是 1。④ 仅适用于顺序表，因为需要按下标随机访问（链表无法高效跨步访问）。

2.2 交换排序

冒泡排序

冒泡排序每趟从后向前（或从前向后）两两比较相邻元素，逆序则交换，每趟将最小（或最大）元素冒泡到表头（或表尾）。若某一趟未发生交换，说明已有序，可提前终止。

C · 冒泡排序（带提前终止）

```
void BubbleSort(int *A, int n) {
    int i, j, temp;
    bool flag;
    for (i = 0; i < n - 1; i++) {
        flag = false;
        for (j = n - 1; j > i; j--) { // 从后向前
            if (A[j-1] > A[j]) {
                temp = A[j-1]; A[j-1] = A[j]; A[j] = temp;
                flag = true;
            }
        }
        if (!flag) return; // 本趟无交换，已有序
    }
}
```

性能：最好 $O(n)$ （已有序，一趟即终止），平均/最坏 $O(n^2)$ 。空间 $O(1)$ 。**稳定。**冒泡排序适用于链表（只需修改指针），且在序列基本有序时效率高。

快速排序（重点）

快速排序是基于**分治**思想的内部排序算法，平均时间复杂度 $O(n \log n)$ ，是实际应用中最快的通用排序之一。其核心是**划分（Partition）**操作：选择一个枢轴（pivot），将序列分为两部分——左半区元素均 $\leq$ pivot，右半区元素均 $>$ pivot，然后对左右半区递归排序。

图 4 快速排序的一次划分（以首元素为枢轴）

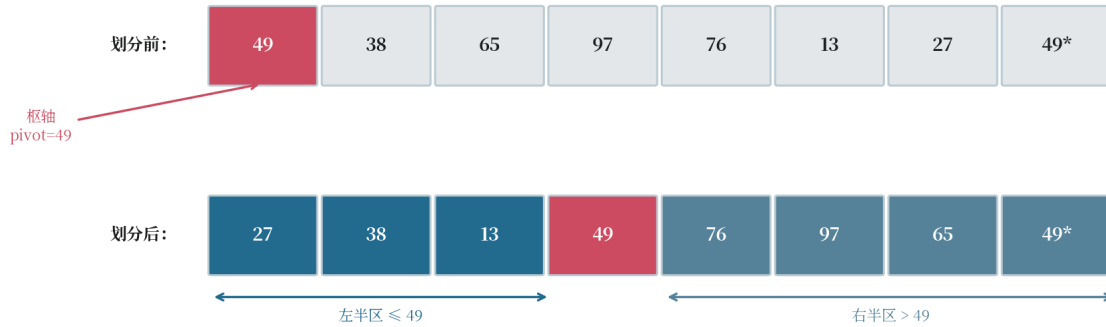


图 4 快速排序的一次划分（以首元素为枢轴）

C · 快速排序（Hoare 划分）

```
int Partition(int *A, int low, int high) {
    int pivot = A[low]; // 取首元素为枢轴
    while (low < high) {
        while (low < high && A[high] >= pivot) high--;
        A[low] = A[high]; // 比枢轴小的移到低端
        while (low < high && A[low] <= pivot) low++;
        A[high] = A[low]; // 比枢轴大的移到高端
    }
    A[low] = pivot; // 枢轴归位
    return low; // 返回枢轴最终位置
}

void QuickSort(int *A, int low, int high) {
    if (low < high) {
        int pivotpos = Partition(A, low, high);
        QuickSort(A, low, pivotpos - 1);
        QuickSort(A, pivotpos + 1, high);
    }
}
```


快速排序核心考点

① **时间复杂度**：平均 $O(n \log n)$ ，最坏 $O(n^2)$ （序列已有序且枢轴选首/尾）。② **空间复杂度**：平均 $O(\log n)$ （递归栈），最坏 $O(n)$ 。③ **不稳定**。④
枢轴选择影响性能：三数取中、随机化可避免最坏情况。⑤
快速排序是内部排序中平均性能最优的算法，但不适合链表（链表无法高效双向扫描）。

力扣题单 · 交换排序与划分思想

题号	题目	难度	对应考点	解题提示
912	排序数组	中等	快排/归并/堆 综合练习	注意快排最坏情况，可用随机化
215	数组中的第 K 个最大元素	中等	快排划分 · TopK	QuickSelect，平均 $O(n)$
75	颜色分类	中等	三路划分（荷兰国旗）	三指针一次遍历
324	摆动排序 II	中等	划分 + 交叉放置	先找中位数再三路划分
280	摆动排序	中等	相邻交换	无需完整排序
70	爬楼梯	简单	（排序热身）	动态规划基础

2.3 选择排序

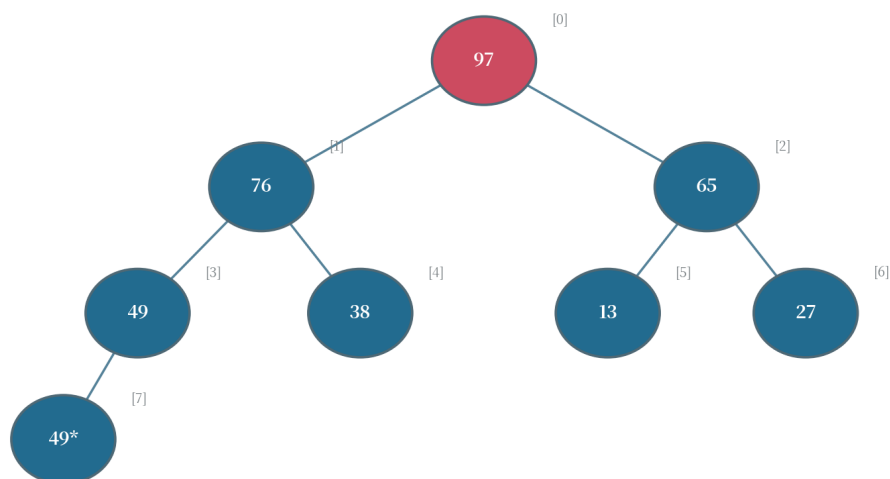
简单选择排序

简单选择排序每趟从无序区选出最小元素，与无序区第一个元素交换，直到无序区为空。无论序列初始状态如何，比较次数始终为 $n(n-1)/2$ ，时间复杂度恒为 $O(n^2)$ 。

堆排序（重点）

堆排序利用堆这种数据结构进行排序。堆是满足堆性质的完全二叉树：大根堆中每个节点值 $\geq$ 其子节点值（根最大）；小根堆反之。堆排序分为两步：① 建堆（将无序序列调整为堆）；② 反复取出堆顶（最大/最小值）并调整堆。

图 5 大根堆的存储结构（数组下标对应树节点）



步骤: ① 输出堆顶 97 (与末尾交换) ② 末尾元素下沉调整 ③ 堆规模 -1, 重复

图 5 大根堆的存储结构（数组下标对应树节点）

C · 堆排序（大根堆，升序）

```

// 调整：将以 k 为根的子树调整为大根堆
void AdjustDown(int *A, int k, int len) {
    int temp = A[k];
    for (int i = 2*k + 1; i < len; i = 2*i + 1) { // 左孩子
        if (i + 1 < len && A[i] < A[i+1]) i++; // 取较大孩子
        if (temp >= A[i]) break; // 已满足堆性质
        A[k] = A[i]; // 孩子上移
        k = i; // 继续向下
    }
    A[k] = temp; // 原值归位
}

void HeapSort(int *A, int n) {
    // 1. 建堆：从最后一个非叶节点向前调整
    for (int i = n/2 - 1; i >= 0; i--)
        AdjustDown(A, i, n);
    // 2. 排序：堆顶与末尾交换，缩小堆并调整
    for (int i = n - 1; i > 0; i--) {
        int t = A[0]; A[0] = A[i]; A[i] = t;
        AdjustDown(A, 0, i);
    }
}
    
```

堆排序核心考点

① **建堆时间复杂度 $O(n)$** （不是 $O(n \log n)$ ，这是常考易错点）。② 排序阶段共 $n-1$ 次调整，每次 $O(\log n)$ ，总 $O(n \log n)$ 。③ **不稳定**。空间 $O(1)$ 。④ 升序用**大根堆**，降序用**小根堆**（堆顶与末尾交换）。⑤ 堆的数组下标：节点 i 的父节点为 $(i-1)/2$ ，左孩子 $2i+1$ ，右孩子 $2i+2$ （0 基址）。⑥ 堆排序**适合 TopK 问题**（维护大小为 K 的堆即可，无需全排序）。

力扣题单 · 堆与 TopK

题号	题目	难度	对应考点	解题提示
215	数组中的第 K 个最大元素	中等	小根堆 · TopK	维护大小为 K 的小根堆
347	前 K 个高频元素	中等	堆 + 哈希表	先统计频次再用堆
295	数据流的中位数	困难	双堆（大根+小根）	大根堆存小半，小根堆存大半
703	数据流中的第 K 大元素	简单	小根堆 · TopK	维护大小为 K 的小根堆
23	合并 K 个升序链表	困难	小根堆 · 多路归并	堆中存每链表当前最小节点
378	有序矩阵中第 K 小的元素	中等	小根堆 · 多路归并	矩阵每行视为一路
1046	最后一块石头的重量	简单	大根堆	每次取最大两块碰撞

2.4 归并排序（Merge Sort）

归并排序也是基于**分治**的排序算法。它将序列递归地分为两半，分别排序后合并。2 路归并排序是最常见的形式：每趟将相邻的两个有序子序列合并为一个有序序列。归并排序的时间复杂度恒为 $O(n \log n)$ （无论最好最坏），且**稳定**，但需要 $O(n)$ 辅助空间。

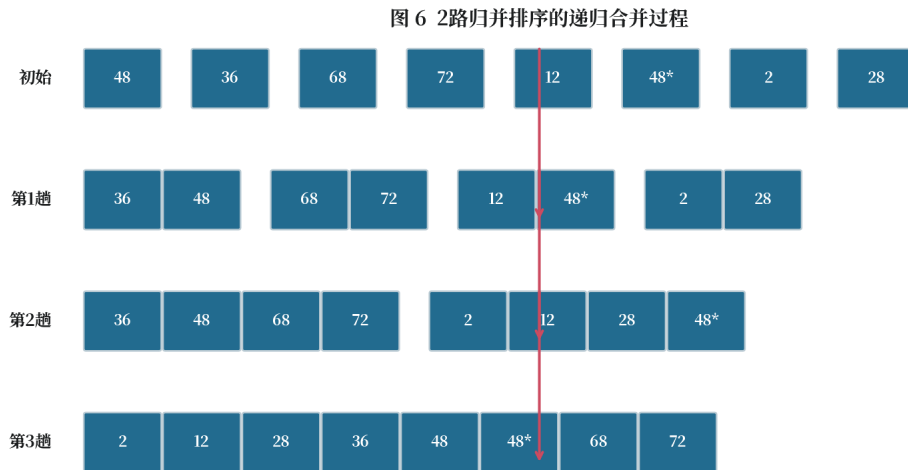


图 6 2 路归并排序的递归合并过程

C · 2 路归并排序

```
int *B; // 辅助数组

void Merge(int *A, int low, int mid, int high) {
    int i, j, k;
    for (k = low; k <= high; k++) B[k] = A[k]; // 复制到辅助数组
    for (i = low, j = mid + 1, k = i; i <= mid && j <= high; k++) {
        if (B[i] <= B[j]) A[k] = B[i++]; // 注意 <= 保证稳定性
        else A[k] = B[j++];
    }
    while (i <= mid) A[k++] = B[i++]; // 处理剩余
    while (j <= high) A[k++] = B[j++];
}

void MergeSort(int *A, int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        MergeSort(A, low, mid); // 左半
        MergeSort(A, mid + 1, high); // 右半
        Merge(A, low, mid, high); // 合并
    }
}
```

归并排序要点

- ① **时间复杂度**: $O(n \log n)$ (恒定, 与初始序列无关)。
- ② **空间复杂度**: $O(n)$ (辅助数组)。
- ③ **稳定** (合并时左半优先可保证稳定性)。
- ④ **适用于链表** (无需随机访问, 只需顺序遍历)。
- ⑤ 归并排序是**外部排序**的基础 (多路归并)。
- ⑥ n 路归并的归并趟数 = $\lceil \log_n k \rceil$, 其中 k 为初始归并段数。

力扣题单 · 归并思想

题号	题目	难度	对应考点	解题提示
88	合并两个有序数组	简单	归并合并操作	从后向前填充避免移动
21	合并两个有序链表	简单	归并 · 链表	迭代或递归
23	合并 K 个升序链表	困难	多路归并	堆或分治两两合并
148	排序链表	中等	归并排序 · 链表	快慢指针找中点 + 归并
315	计算右侧小于当前元素的个数	困难	归并 · 逆序对	归并过程中统计逆序对
327	区间和的个数	困难	归并 · 逆序对变体	前缀和 + 归并统计
493	翻转对	困难	归并 · 逆序对变体	归并过程中统计

2.5 基数排序 (Radix Sort)

基数排序是一种**非比较**的排序算法。它借助"分配"与"收集"两种操作，按关键字各位的权重依次排序。最低位优先 (LSD) 是最常见方式：从最低位到最高位，每位做一次分配（按该位值分到 10 个队列）和收集（依次收集各队列）。

性能：时间复杂度 $O(d \times (n+r))$ ，其中 d 为关键字位数， r 为基数（如十进制 $r=10$ ）。空间复杂度 $O(r)$ （队列）。**稳定。**基数排序适用于关键字位数少且可分解的场景（如手机号、身份证号排序），但**不适用于浮点数和字符串**（需特殊处理）。

基数排序 vs 计数排序 vs 桶排序

三者都是非比较排序：**计数排序**适用于关键字范围小的整数 ($O(n+k)$)；**桶排序**将数据分到若干桶后桶内排序（平均 $O(n)$ ）；**基数排序**按位分配收集 ($O(d(n+r))$)。408

主要考查基数排序，需理解"分配-收集"过程与稳定性。

2.6 外部排序 (External Sort)

当数据量过大无法全部装入内存时，需进行外部排序。外部排序通常采用**多路归并**：① 将大文件分块读入内存，每块内部排序后写回外存，形成**初始归并段**；② 对这些归并段进行多路归并，直到合并为一个有序文件。

败者树 (Loser Tree)

多路归并中，每次需从 k 路归并段的当前最小值中选出全局最小。若用简单比较，每次需 $k-1$ 次比较。**败者树**是一棵完全二叉树，将每次比较的"败者"（较大者）记录在父节点，"胜者"（较小者）继续上浮。败者树的高度为 $\lceil \log_2 k \rceil$ ，因此每次选最小只需 $\lceil \log_2 k \rceil$ 次比较，比简单比较更高效。

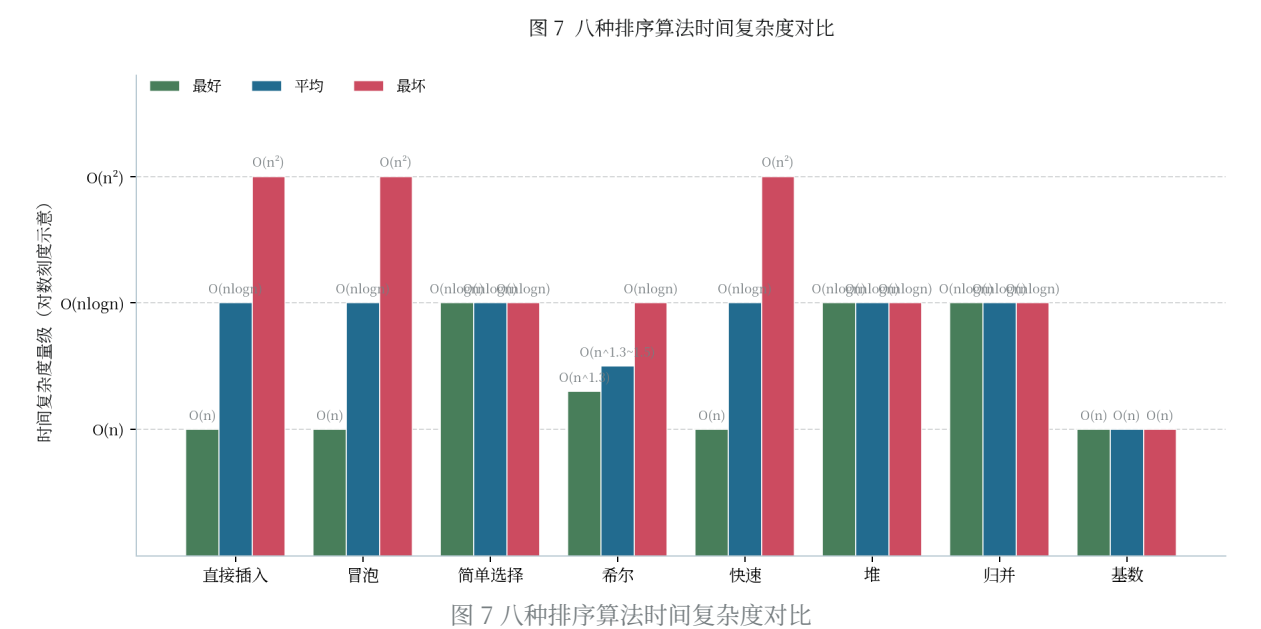
置换-选择排序（生成初始归并段）

置换-选择排序用于生成初始归并段，使归并段长度更长（平均为内存工作区大小的 2 倍），从而减少归并段数。其核心是：维护一个内存工作区（大小为 w ），每次输出工作区最小值（要求大于上一个输出值），并从外存读入下一个元素补充。当工作区所有元素都小于上一个输出值时，归并段结束，开始新段。

外部排序考点

① 归并趟数 = $\lceil \log_k r \rceil$ ，其中 k 为归并路数， r 为初始归并段数。② 增加 k 可减少趟数，但会增加每趟比较次数（败者树可优化）。③ 减少初始归并段数 r （用置换-选择排序）也能减少趟数。④ I/O 次数 = 趟数 $\times$ 每趟读写次数，是外部排序的主要代价。

2.7 排序章节力扣题单汇总



知识点	题号清单	建议优先级
插入排序	912, 147（链表插入排序）	★★★★ 推荐
希尔排序	（力扣无直接题，理解原理）	★★ 理解
冒泡排序	（基础题，理解原理即可）	★★ 理解
快速排序	912, 215, 75, 324	★★★★★ 必刷
堆排序/TopK	215, 347, 295, 703, 23, 378, 1046	★★★★★ 必刷
归并排序	88, 21, 23, 148, 315, 327, 493	★★★★★ 必刷
基数排序	（力扣无直接题，理解原理）	★★ 理解
外部排序	（力扣无对应题，掌握手算）	★★ 理解

第三章 综合练习与真题对照

本章将查找与排序的知识点与 408 历年真题题型对照，帮助考生把握命题规律。408 数据结构的算法设计题（综合应用题）通常为 10-15 分，要求用 C 语言描述算法并分析复杂度。从历年真题看，查找与排序是算法设计题的两大核心来源。

3.1 高频真题题型归纳

题型	考查知识点	代表真题	力扣对应
散列表构造 + ASL	散列函数、冲突解决、ASL 计算	2010、2013、2018	146, 705
BST 构造与遍历	BST 插入、中序遍历、平衡	2011、2017	108, 450
B 树插入/删除	B 树分裂、合并、调整	2014、2016	—
快排划分过程	Partition 手工模拟	2010、2012、2019	215, 912
堆排序调整	建堆、AdjustDown 手工模拟	2011、2015	215, 347
归并排序过程	归并趟数、归并段	2013、2018	88, 148
排序算法综合	稳定性、复杂度、适用性	2010、2014、2020	912

3.2 算法设计题答题模板

408 算法设计题的评分标准通常为：算法思想（4 分）+ 代码实现（8 分）+ 复杂度分析（1 分）。建议按以下模板作答，确保拿到全部步骤分。

答题四步法

第一步·算法思想（文字描述）：用 2-3

句话说明算法的核心思路，包括数据结构选择、关键操作步骤、终止条件。

第二步·数据结构定义：若题目未给出，需自定义结构体（如链表节点、树节点）。

第三步·代码实现：用 C 语言写出完整函数，注意：① 函数签名规范；② 边界条件处理；③ 变量命名清晰；④ 必要注释。

第四步·复杂度分析：写出时间复杂度 $O(?)$ 与空间复杂度 $O(?)$ ，并简要说明依据。

3.3 综合练习题（模拟真题）

练习 1：散列表构造与 ASL 计算

题目：给定关键字序列 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key) = key \bmod 13$ ，采用链地址法处理冲突。（1）画出散列表；（2）计算查找成功和查找失败的 ASL（假设等概率）。

解题思路

- ① 依次计算每个关键字的散列地址： $H(19)=6$, $H(14)=1$, $H(23)=10$, $H(1)=1$, $H(68)=3$, $H(20)=7$, $H(84)=6$, $H(27)=1$, $H(55)=3$, $H(11)=11$, $H(10)=10$, $H(79)=1$ 。
- ② 链地址法：同一地址的关键字挂同一链表，链表内按插入顺序排列。
- ③ 查找成功 $ASL = \Sigma(\text{每条链表深度})/12$ ；查找失败 $ASL = \Sigma(\text{每条链表长度})/13$ （含空链表）。

练习 2：快速排序划分手工模拟

题目：给定序列 {49, 38, 65, 97, 76, 13, 27, 49*}，以首元素 49 为枢轴，写出快速排序一次划分后的序列，并指出枢轴最终位置。

解题思路

- ① 划分过程：枢轴 49，high 从右扫描找 <49 的元素（27）移到 low；low 从左扫描找 >49 的元素（65）移到 high；重复直到 $low==high$ 。
- ② 划分结果：{27, 38, 13, 49, 76, 97, 65, 49*}，枢轴最终位置为下标 3。
- ③ 左半区 {27, 38, 13} 均 ≤ 49 ，右半区 {76, 97, 65, 49*} 均 > 49 。

练习 3：堆排序建堆过程

题目：给定序列 {49, 38, 65, 97, 76, 13, 27, 49*}，将其建立为大根堆，写出建堆后的序列，并画出对应的树形结构。

解题思路

- ① 从最后一个非叶节点（下标 $n/2-1=3$ ，即 97）向前调整。
- ② 调整 97：已是叶子（相对子树），无需调整。
- ③ 调整 65：子节点 13、27 均小于 65，无需调整。
- ④ 调整 38：子节点 97、76， $97>38$ ，交换 38 与 97。交换后 38 的子节点 49*， $49*>38$ ，再交换。
- ⑤ 调整 49（根）：子节点 97、65， $97>49$ ，交换。继续调整直到满足堆性质。
- ⑥ 最终大根堆：{97, 76, 65, 49, 38, 13, 27, 49*}（具体结果取决于调整细节）。

附录 排序算法复杂度速查表

本附录汇总 408 考纲要求的七大内部排序算法的核心指标，建议考前反复默写至烂熟于心。记忆口诀：“插冒选快堆归基”（直接插入、冒泡、简单选择、快速、堆、归并、基数）。

算法	最好时间	平均时间	最坏时间	空间	稳定性	适用
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	顺序表/链表
折半插入	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	顺序表
希尔排序	—	$O(n^{1.3})$	$O(n^2)$	$O(1)$	不稳定	顺序表
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	顺序表/链表
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	$O(\log n)$	不稳定	顺序表
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	顺序表/链表
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定	顺序表
归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定	顺序表/链表
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	稳定	顺序表/链表

A.2 查找算法 ASL 速查表

查找算法	查找成功 ASL	查找失败 ASL	备注
顺序查找	$(n+1)/2$	$n+1$	等概率，带哨兵
折半查找	$\approx \log_2(n+1) - 1$	$\approx \log_2 n$	判定树高 $\lceil \log_2 n \rceil + 1$
分块查找	$(s^2+2s+n)/(2s)$	—	$s=\sqrt{n}$ 时最优， $ASL=\sqrt{n}+1$
BST（平衡）	$\approx \log_2 n$	$\approx \log_2 n$	退化为链表时 $O(n)$
AVL	$\approx \log_2 n$	$\approx \log_2 n$	严格平衡，高 $O(\log n)$
散列·线性探测	$\approx (1+1/(1-\alpha))/2$	$\approx (1+1/(1-\alpha)^2)/2$	α 为装填因子
散列·链地址	$\approx 1+\alpha/2$	$\approx \alpha+e^{-\alpha}$	无堆积，删除方便

A.3 关键公式与结论速记

- 折半查找判定树高 = $\lceil \log_2 n \rceil + 1$ （ n 为元素数）。
- m 阶 B 树节点关键字数：根 ≥ 1 ，非根非叶 $\lceil m/2 \rceil - 1 \sim m - 1$ 。
- B 树高度： $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil}((n+1)/2) + 1$ 。

- **堆排序建堆**: $O(n)$; 排序阶段: $O(n \log n)$; 总: $O(n \log n)$ 。
 - **快排最坏**: 序列有序 + 枢轴选首/尾 $\rightarrow O(n^2)$, 空间 $O(n)$ 。
 - **归并趟数** (2 路) = $\lceil \log_2 r \rceil$, r 为初始归并段数。
 - **k 路归并趟数** = $\lceil \log_k r \rceil$, 败者树每次比较 $\lceil \log_2 k \rceil$ 次。
 - **散列表装填因子** α = 记录数 / 表长, α 越大 ASL 越大。
 - **稳定排序**: 插入、冒泡、归并、基数。**不稳定**: 希尔、快排、选择、堆。
 - **链表适用**: 插入、冒泡、归并、基数。**仅顺序表**: 折半插入、希尔、快排、堆。
-